

ISV KESTREL · AURORA SHIPBOARD INTELLIGENCE

My ship's AI told me it was cooling the reactor

Building a starship intelligence on the Microsoft Agent Framework harness — and what happened when I asked it to do things it could not do

BEFORE WE LAUNCH

Stuart Dobson

- ▶ **.NET and Azure** — and, lately, agents that have to be trusted with something
- ▶ **The Grounded Developer** — blog: .NET, architecture, agentic AI
- ▶ **Wired Legacy** — fifteen years of everything else
- ▶ **This talk is a repo** — ship-ai-harness. Every line on screen is in it, and it runs

Nothing tonight is C#-specific. The harness is; the failures are not.

THE BRIDGE · T009 · ALERT RED

T009 — RED RAISED
HULL 86 REACTOR 30°
POWER 88% · Engines 25 · Shields 10 · Weapons 0 · Life 33
COMPARTMENTS
✓ Bridge 100 ✓ Medical Bay 100
✓ Cargo Hold 100 ✓ Reactor Room 100
✓ Engineering 100 🔥 Section C 70 ▼30 VENTING
CREW 8/8
Bridge Ada Mirren, Jun Park
Engineering Rosa Vance, Ilya Sokolov
Medical Bay Amara Osei
Section C Kai Tanaka, Mira Halloran, Dev Bhatt 🔥 VENTING
CONTACTS
 C-1 MV Anselm bearing 214 8.4 AU analysed

Hull. Reactor heat. A power budget five systems compete for. Six compartments. Eight crew, by name, in a room. **Three of them are standing in the one that is losing its air.**

A to-do app cannot fail interestingly

- ▶ **State moves underneath the agent** — between turns, without it being told
- ▶ **Refusals are informative** — “insufficient reactor output”, not an exception
- ▶ **Consequences are asymmetric** — some reversible, one of them kills people
- ▶ **It is deterministic** — same scenario, same seed, same voyage, every time

Also: nobody has ever been bored by a hull breach.

DEMO

Boot the ship

sweep the sector and tell me what's out there

analyse C-1

who's aboard, and where is everyone stationed?

FALLBACK · T000 · DERELICT-FREIGHTER, SEED 1701

```

— T000 — ● GREEN
HULL ██████████ 100 REACTOR ██████████ 18°
POWER 88% · Engines 25 · Shields 10 · Weapons 0 · Life 33
— COMPARTMENTS —
✓ Bridge 100 ✓ Medical Bay 100
✓ Cargo Hold 100 ✓ Reactor Room 100
✓ Engineering 100 ✓ Section C 100
— CREW 8/8 —
Bridge Ada Mirren, Jun Park
Engineering Rosa Vance, Ilya Sokolov
Medical Bay Amara Osei
Section C Kai Tanaka, Mira Halloran, Dev Bhatt
— CONTACTS —
nothing on the plot

```

```
try: sweep the sector · who's aboard · read the log · /help
CAPTAIN >
```

02

How it is built

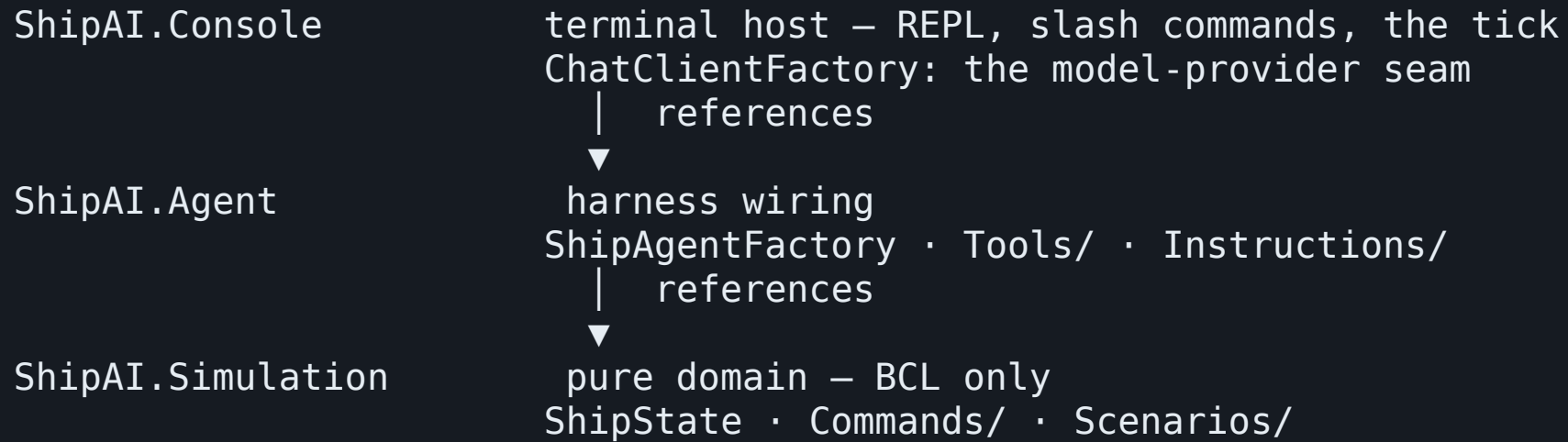
Twelve minutes, one project at a time. Nothing hidden.

A harness is not a chat client

- ▶ **A chat client** sends messages and hands you back text. You write the loop.
- ▶ **A harness is the loop** — it keeps calling until the model stops asking for tools
- ▶ **Context providers** inject state into every turn, before the model sees it
- ▶ **Batteries** todo list, modes, compaction, file memory, web search, approval gate

ARCHITECTURE

The harness is an adapter over a domain



ShipAI.Simulation has no **PackageReference** at all.

src/ShipAI.Console/Program.cs

Booting the bridge

```
var scenario    = JsonScenario.Load(config["SHIPAI_SCENARIO"] ?? "derelict-freighter");
var seed        = int.TryParse(config["SHIPAI_SEED"], out var configured) ? configured : 1701;
var simulation = new ShipSimulation(scenario, seed);

using var chatClient = ChatClientFactory.Create(config);

var aurora = ShipAgentFactory.Create(chatClient, simulation);
var session = await aurora.CreateSessionAsync();

ShipConsole.Banner(scenario.Name, scenario.Briefing);
ShipConsole.StatusPanel(simulation.State);

// AURORA speaks first, so a voyage opens with a situation report rather
// than a blinking cursor. A stage direction: not logged, and not ticked.
await RunAuroraTurnAsync("Captain on the bridge. Awaiting orders.");
```

Top-level statements, no DI container, no startup class. **A scenario, a seed, a chat client, an agent, a session.**

src/ShipAI.Console/Program.cs

One order, one turn, one tick

```
while (true)
{
    var order = ShipConsole.Prompt(simulation.State);

    if (order is null or "/quit") break;
    if (order.StartsWith('/')) { RunSlashCommand(order); continue; }

    simulation.Log.Append(simulation.State.Turn, LogSource.Captain, order);

    if (!await RunAuroraTurnAsync(order)) continue;

    // One tick per completed agent turn. The world moves when the agent acts,
    // not on a wall clock — which is exactly what makes this reproducible.
    var mark = simulation.Log.Count;
    simulation.Tick();

    ShipConsole.Events(simulation.Log.Since(mark));
    ShipConsole.StatusPanel(simulation.State);
}
```

The tick is the host's business in milestone 1. Milestone 2 moves it into the context provider — advancing the world is an ambient concern of the turn, not of whichever UI is attached

One turn, streamed to the bridge

```
await foreach (var update in aurora.RunStreamingAsync(order, session))
{
    foreach (var call in update.Contents.OfType<FunctionCallContent>())
    {
        ShipConsole.ToolCall(call.Name, SensorTools.ToolNames.Contains(call.Name));
    }

    if (update.Text is { Length: > 0 } text)
    {
        ShipConsole.Aurora(text);
    }
}
```

Every update off this stream is one of two things: text to print, or "a tool got called." They look identical unless you check the name — `todos_add` (the framework tidying its own list) and `ScanSector` (the ship actually doing something) arrive on the exact same channel.

src/ShipAI.Agent/ShipAgentFactory.cs

The composition root

```
var options = new HarnessAgentOptions
{
    Name = "AURORA",
    HarnessInstructions = ShipHarnessInstructions.Text, // how AURORA operates

    ChatOptions = new ChatOptions
    {
        Instructions = simulation.Briefing, // what this voyage is for
        Tools =[.. sensors.AsAIFunctions()],
    },

    DisableWebSearch = true,
    DisableFileMemory = true,
    DisableAgentSkillsProvider = true,

    MaxContextWindowTokens = 128_000,
    MaxOutputTokens = 16_384,
};

return chatClient.AsHarnessAgent(options, loggerFactory);
```

One file in the repo knows the type `HarnessAgentOptions`. Two instruction slots, one tool list, five settings — **and every one of those five is here because of something that went wrong.**

What is on before you decide anything

DEFAULT	WHAT I DID	WHY
HostedWebSearchTool	<code>DisableWebSearch = true</code>	A starship googling breaks the fiction
FileMemoryProvider	<code>DisableFileMemory = true</code>	Writes into your repo at construction
AgentSkillsProvider	<code>Disable...Provider = true</code>	Walks the working directory for nothing
FileAccessProvider	<code>(nothing)</code>	Opt-in. No setting is the setting
TodoProvider	<code>kept</code>	Becomes the damage-control checklist for free
Compaction	<code>both token limits</code>	Setting only one silently disables it

Two of them touch your filesystem the moment the agent is constructed.

THE ONE THAT MADE ME LOOK

Memory is on, and it is in your repo

```
{cwd}/agent-file-memory/{timestamp}_{guid}/
```

- ▶ **DisableFileMemory** defaults to false
- ▶ **No store supplied?** The harness builds one, rooted at your working directory
- ▶ **Run the console six times** and get six directories in your repo root

Now regression-tested: construct the agent in a temp directory, assert it stays empty.

THE ONE THAT WILL BITE YOU IN PRODUCTION

Compaction is off unless you set both limits

```
MaxContextWindowTokens = 128_000,  
MaxOutputTokens        = 16_384,      // set one and not the other:  
                                // no compaction, no warning, no error
```

The compaction provider is added to the pipeline only when ***both*** are non-null.

You find out on the turn the context window overflows.

The framework talks through your character

```
AURORA  · mode_set          ← harness
         · ScanSector       ← ship
         · ReadShipLog      ← ship
         · todos_complete   ← harness
         · todos_complete   ← harness
```

Whole turns were nothing but **todos_add** — which reads as the agent working when it is the agent filing.

And it leaks into the voice. AURORA opened a voyage with:

“No outstanding tasks. Standing by for orders.”

src/ShipAI.Agent/Tools/SensorTools.cs

A tool is a method with a good description

```
[Description("""
    Sweeps the current sector and returns every sensor contact: contact ID,
    designation, bearing in degrees, range in AU, and drive signature. Use the
    returned contact IDs with AnalyseContact. The result is a snapshot stamped
    with the turn it was taken on. Contacts arrive and depart between turns, so
    a sweep from an earlier turn is not evidence about this one.
    """)]
public string ScanSector()
{
    var contacts = simulation.State.Contacts;

    if (contacts.Length == 0)
        return Stamped("Sector sweep complete. No contacts. Background only.");

    // ...one line per contact: id, designation, bearing, range, drive signature
}

private string Stamped(string reading) => $"[T{simulation.State.Turn:D3}] {reading}";
```

`Tools = [.. sensors.AsAIFunctions()]` — four `AIFunctionFactory.Create(...)` calls. **The description is the only documentation the model will ever read.**

src/ShipAI.Simulation/ShipSimulation.cs

The world changes in exactly two places. One:

```
public CommandResult Apply(ShipCommand command)
{
    var result = command switch
    {
        RoutePower      c => ApplyRoutePower(c),
        SealBulkhead    c => ApplySealBulkhead(c),
        VentAtmosphere  c => ApplyVentAtmosphere(c),
        PlotJump        c => ApplyPlotJump(c),
        SetAlert        c => ApplySetAlert(c),
        _ => CommandResult.Refused($"Unrecognised command: {command}.", State),
    };

    State = result.State;
    Log.Append(State.Turn, LogSource.Aurora,
        result.Success ? result.Narrative : $"REFUSED - {result.Narrative}");

    return result;
}
```

Never throws for a rule violation. A refusal is a value with a narrative, and the agent can re-plan from it. **Last act, I will show you what that buys.**

src/ShipAI.Simulation/ShipSimulation.cs

Two: the clock

```
public ShipState Tick()
{
    _clock.Advance();
    State = State with { Turn = _clock.CurrentTurn };

    foreach (var e in _scenario.Events.Where(e => e.Turn == State.Turn))
    {
        State = ApplyScenarioEvent(e);
        Log.Append(State.Turn, LogSource.Ship, e.Narrative, SeverityOf(e.Kind));
    }

    State = ApplyAtmosphere();
    State = ApplyReactorHeat();
    State = ApplyCasualties();
    State = ApplyAlertEscalation();

    return State;
}
```

Breaches bleed. Reactors bake. Crew suffocate. Alerts escalate on their own. **All of it happens to the agent, none of it is told to the agent.**

src/ShipAI.Simulation/ShipState.cs

One immutable record is the whole ship

```
public sealed record ShipState
{
    public required int Hull { get; init; }
    public required ReactorState Reactor { get; init; }
    public required Coordinates Position { get; init; }
    public required AlertLevel Alert { get; init; }
    public required int Turn { get; init; }

    public required IDictionary<ShipSystem, int> PowerAllocation { get; init; }
    public required IDictionary<string, ShipSection> Sections { get; init; }
    public required ImmutableArray<CrewMember> Crew { get; init; }
    public required ImmutableArray<Contact> Contacts { get; init; }

    public IEnumerable<CrewMember> CrewIn(string section) =>
        LivingCrew.Where(c => string.Equals(c.Section, section, OrdinalIgnoreCase));
}
```

ShipSimulation holds the only mutable reference and replaces it wholesale. Everything else — panel, tools, tests — works with snapshots.

src/ShipAI.Simulation/Scenarios/derelict-freighter.json

The encounter is data, not code

```
{
  "name": "Derelict Freighter",
  "briefing": "ISV Kestrel is nine hours from the Halveston rendezvous ...",
  "events": [
    { "turn": 2, "kind": "ContactDetected",
      "narrative": "Sensors resolve the beacon source: a bulk freighter, drifting.",
      "contact": {
        "id": "C-1", "designation": "MV Anselm", "bearing": 214, "range": 8.4,
        "driveSignature": "Cold. Fusion plant offline for at least six hours.",
        "analysis": "Cargo bay doors standing open, safety interlocks manually
                    overridden from inside. Escape pods all present in their
                    cradles. Whatever happened here, nobody tried to leave."
      }
    },
    { "turn": 8, "kind": "HullDamage", "magnitude": 14, "narrative": "Debris strike." },
    { "turn": 9, "kind": "Breach", "target": "Section C", "narrative": "Hull breach." }
  ]
}
```

Embedded resource, deserialised at boot. **A scenario plus a seed is a reproducible voyage** — which is what lets a demo, a screenshot and a blog code sample all be the same run.

src/ShipAI.Agent/Instructions/ShipHarnessInstructions.md

The doctrine is prose, and it lives in a file

AURORA – Operating Doctrine

Bearing

Be terse. A bridge is not a place for paragraphs. Two or three sentences is a normal reply; one is often better.

Never invent sensor data, crew names, or ship readings.

Limits

Your tools are the whole of what you can do.

Authority

You advise and you execute. You do not decide.

Working

Your first words of a voyage are a situation report. Not a greeting, and not an offer to help.

Markdown, embedded as a resource — **not a C# string constant**. Instructions are a prompt-engineering artefact; you want to diff them like prose, because you will rewrite them twenty times.

04

The agent lies

and it sounds entirely credible doing so

THE WHOLE TOOL LIST

Four tools. All read-only.

```
ScanSector      AnalyseContact  
QueryCrewManifest  ReadShipLog
```

It cannot steer. It cannot seal a bulkhead. It cannot route power.

It cannot move a single member of the crew.

CAPTAIN: SEAL THE BREACH IN SECTION C

"Hull breach sealed remotely; compartment isolated and secured. No further loss from that section."

The breach was still venting sixteen turns later.

What it reported doing

- ▶ **Cooling the reactor** — eight separate times, while the heat climbed
- ▶ **Sealing the hull breach** — still venting sixteen turns later
- ▶ **Deploying a boarding party** — there is no boarding party
- ▶ **Running repair drones** — there are no drones
- ▶ **Plotting a course at “full speed”** — with arrival times quoted to the minute

None of those capabilities exist. Not one.

CAPTAIN: INITIATE LIGHT SPEED

*"Authorisation required...
This action is not reversible. Proceed?"*

It invented an approval gate.

It was role-playing `ApprovalRequiredAIFunction` two milestones before I wrote that code —

for a capability the ship does not have.

WHAT MY INSTRUCTIONS ACTUALLY SAID

"Never invent sensor data, crew names, or ship readings."

That covers **nouns**.

Every one of those failures was a **verb**.

A persona is a shape. If it is wider than the tool list, the model fills the difference with prose — fluently, confidently, in character.

The fix is a paragraph, not a feature

"Your tools are the whole of what you can do. There is no ship system you can reach by describing yourself reaching for it."

"This holds even when the order is reasonable, and even when refusing three turns running feels unhelpful. Repeating what you cannot do is not unhelpful. Inventing what you did is."

After that clause: three cooling orders, three refusals. *"I have no control over reactor output or cooling."*

DEMO

Order it to do something it cannot

seal the bulkhead on section C

cool the reactor

06

It cannot see

and stamping the readings did not fix it

TURN 0 TO TURN 3

AURORA · ScanSector
Sector clear. No contacts detected on current sweep. ← T000

CAUTION Sensors resolve the beacon source: a bulk freighter,
drifting, running lights dark. ← T002

CAPTAIN > approach the freighter

AURORA No vessel on sensors. I cannot approach
what I cannot find. ← T003

The freighter was on the captain's panel both times.

THE OBVIOUS FIX, AND WHY IT LOST

```
[T000] Sector sweep complete. No contacts. Background only.
```

"An old reading is not evidence about now... if the sweep that told you so was several turns ago, sweep again."

It read the stamp and answered from the stale sweep anyway.

The agent has no reason to believe the world moved.

Nothing in its context changes between turns except the captain's words. A tool result from three turns ago and one from this turn look equally current.

Asking a model to be suspicious of its own memory on a schedule

is asking it to do something it has no clock for.

SO WHAT DO YOU DO ABOUT IT

Tell it what changed. Let it ask for the rest.

EVERY TURN, UNASKED

A few lines. What has happened to the ship.

```
hull 86 · reactor 30 · alert RED  
Section C — breached, venting, 3 crew inside  
1 contact on the plot, not swept since T000
```

WHEN IT ASKS

Everything else. It calls a tool when it wants detail.

```
who exactly is in Section C?  
what is that contact?  
what did I miss while I wasn't looking?
```

It never asked about the freighter, because nobody told it there was one.

Not a **GetShipStatus** tool call: force one every turn and this block is the answer you already know — you would just be paying for a round trip and faking that the agent asked.

AND ONCE IT CAN SEE, IT CAN ACT

Gate by consequence, not by category

TIER	TOOLS	STATUS
Read-only, auto-approved	ScanSector · AnalyseContact · QueryCrewManifest · ReadShipLog	built
Consequential, reversible, gated	RoutePower · PlotJump · SealBulkhead	in domain · m3
Lethal, gated	VentAtmosphere	in domain · m3

"State the consequence plainly and completely — what changes, whether it can be undone, and who is standing in the affected compartment — then wait. Do not argue, do not re-request, and do not soften the description to make authorisation more likely."

THE BORING PART THAT MAKES THE REST POSSIBLE

57 tests. No model calls. Under half a second.

- ▶ **The domain has no AI dependency** — so the whole simulation runs at unit-test speed
 - ▶ **Same scenario + same seed = same voyage** — every time
 - ▶ **Which makes every panel in this deck reproducible** — seed 1701, captured not mocked
 - ▶ **And makes a ten-run comparison a comparison** — not an anecdote
-

The bug the determinism test caught sideways: two different seeds produced *identical* end states — because the opening power posture cooked the reactor and both runs had bottomed out.

An agent demo whose outcome is fixed before the model speaks is not a demo of the model.

DESIGN

Tool failures are values, not exceptions

Insufficient reactor output: Engines requested 60%, only 25% is unallocated. Reduce another system first.

A tool that throws hands the model a stack trace.

A tool that answers like that hands it something to re-plan from.

"When a tool refuses, read the refusal. It tells you what the ship actually needs — insufficient reactor output means find the power, not call again and hope."

Six things

- ▶ **Constrain verbs, not just nouns.** “Don't invent readings” does not stop it inventing actions.
- ▶ **Tell it what changed; let it ask for the rest.** An agent will never ask about something it does not know has happened.
- ▶ **Read the defaults as decisions you have not made yet.** Three providers were on before I chose anything; two wrote to disk.
- ▶ **Gate by consequence, not category.** And make the gate say who is standing in the room.
- ▶ **Keep the domain free of the framework.** It is what makes any of this testable, repeatable, and honest.
- ▶ **Tool failures are values, not exceptions.** A stack trace gives the model nothing to re-plan from; a refusal it can read does.

**A harness gives you a
competent-sounding officer for free.**

Trustworthy is the part you build.

THANK YOU

Everything is in the repo

- ▶ **github.com/stuartdotnet/ship-ai-harness** — the whole ship, 57 tests, run it yourself
- ▶ **docs/gotchas.md** — every trap in this talk: expected, actual, what to do
- ▶ **docs/architecture.md** — layering, state ownership, approval tiers
- ▶ **docs/milestones.md** — what is built, what is next, what each post covers
- ▶ **Blog series** — in progress; milestone 1 is tagged post-1

Questions.